Eleventh Implementation Iteration: Space Allocation

with External CSV-Based Initial Data Loading

Software Design / Iteration Report

Name: Najib Attar
Date: March 2026

1. Iteration Goal

The goal of this eleventh implementation iteration is to improve the prototype by loading initial users and spaces from external CSV files instead of defining all sample data directly inside the `main` function. In previous iterations, the system logic was tested using hardcoded sample objects created manually in code.

In this iteration, the prototype reads initial data from external files. This change improves maintainability, separates sample data from application logic, and prepares the system for future integration with persistent storage such as a database.

2. Changes from Iteration 10

Compared with the tenth iteration, the following changes were introduced:

- Added external CSV files for initial sample data
- Added a simple `DataLoader` component to load users from file
- Added a simple `DataLoader` component to load spaces from file
- Updated the prototype so that `main` uses loaded data instead of manually constructed users and spaces
- Preserved the same request types:
  - `OneTimeRequest`
  - `RecurringRequest`
- Preserved the same supported space types:
  - `Classroom`

  - `Laboratory`
  - `MeetingRoom`

- Preserved the same rules:
  - `AvailabilityRule`
  - `CapacityRule`
  - `FeatureRule`
  - `StatusRule`
  - `LocationRule`

3. Scenario

The prototype starts by loading initial users and spaces from external CSV files. After loading the data, the system creates one-time and recurring requests using the loaded objects and evaluates them according to the existing rules.

The system checks whether the requested space matches the required building, whether the required feature is supported, whether the space is available, whether the participant count fits the capacity, and whether the requested time slot or all requested occurrences are available.

A one-time request is approved if its single occurrence passes all rules. A recurring request is approved only if all occurrences pass all rules. Otherwise, the request is rejected with a stored rejection reason.

4. Modules and Their Tasks

A. Data Loading Module

- Read users from an external CSV file
- Read spaces from an external CSV file
- Create `User`, `Classroom`, `Laboratory`, and `MeetingRoom` objects from the loaded data
- Return the loaded objects to the prototype

B. Request Management Module

- Create and store `OneTimeRequest` or `RecurringRequest`

- Use loaded users and spaces instead of manually created sample objects
- Store request details including required feature, required building, and rejection reason when applicable
- Send the request to the Rule Evaluation Module

C. Resource Management Module

- Manage all supported space types
- Provide information for `Classroom`, `Laboratory`, and `MeetingRoom`
- Provide space identity, type, name, capacity, supported features, availability status, and building information

D. Rule Evaluation Module

- Apply `CapacityRule`
- Apply `FeatureRule`
- Apply `StatusRule`
- Apply `LocationRule`
- Apply `AvailabilityRule`
- Store a short rejection reason when one of the rules fails
- Return the final decision: Approved or Rejected

E. Allocation Management Module

- Create one allocation for a one-time request if all rules pass
- Create multiple allocations for a recurring request if all occurrences pass
- Update request status to Approved or Rejected

5. Iteration Comparison

Iteration	**Request Types**	**Resource Types**	**Rules Implemented**
Iteration 1	OneTimeRequest	Classroom	AvailabilityRule
Iteration 2	OneTimeRequest	Classroom	AvailabilityRule + CapacityRule
Iteration 3	OneTimeRequest	Classroom + Laboratory	AvailabilityRule + CapacityRule
Iteration 4	OneTimeRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule
Iteration 5	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule
Iteration 6	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule
Iteration 7	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule
Iteration 8	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule + StatusRule
Iteration 9	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule + StatusRule
Iteration 10	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule + StatusRule + LocationRule
Iteration 11	OneTimeRequest + RecurringRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule + FeatureRule + StatusRule + LocationRule

6. Updated Class Diagram

Figure 1 shows the updated simplified class diagram for the eleventh implementation iteration.

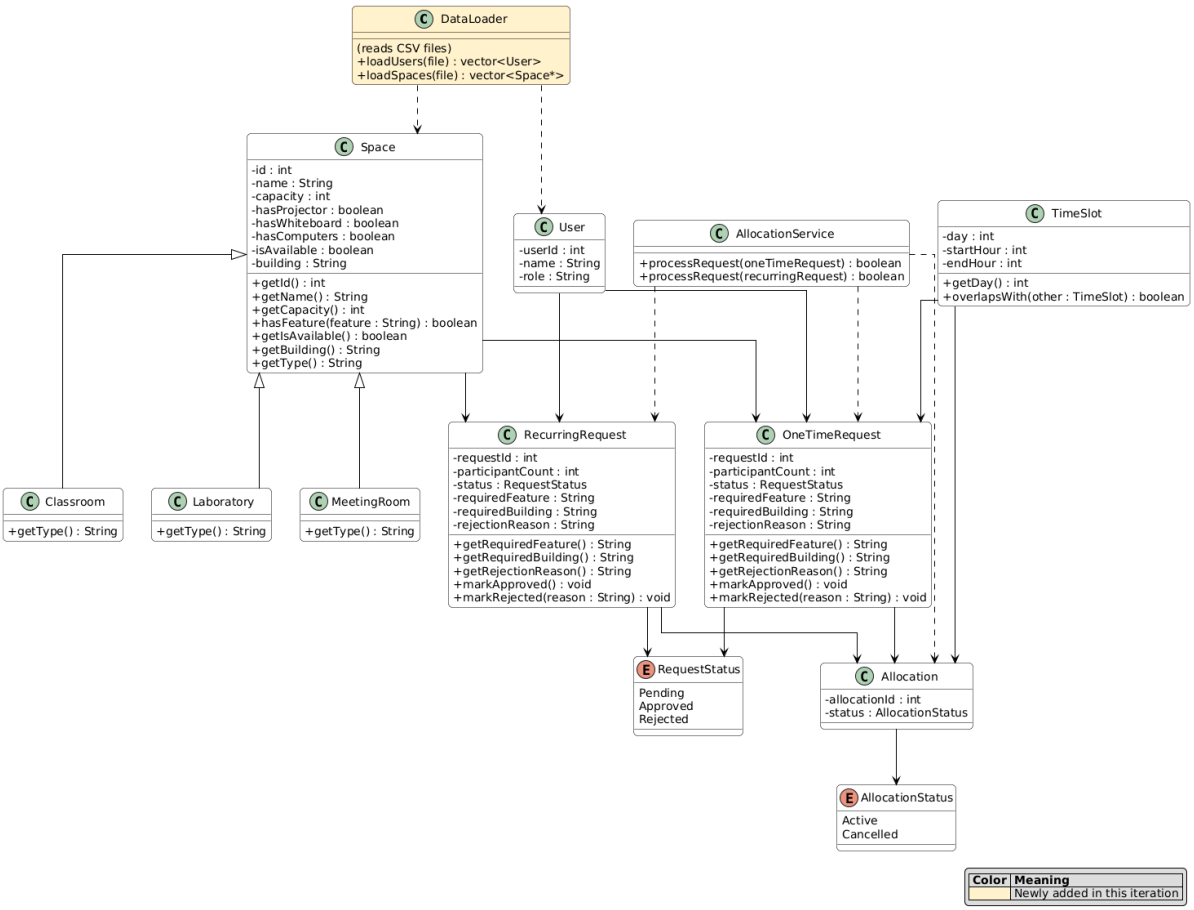

Figure 1: Updated Simplified Class Diagram

7. Updated Workflow Diagram

Figure 2 shows the workflow for the eleventh iteration of the space allocation process.

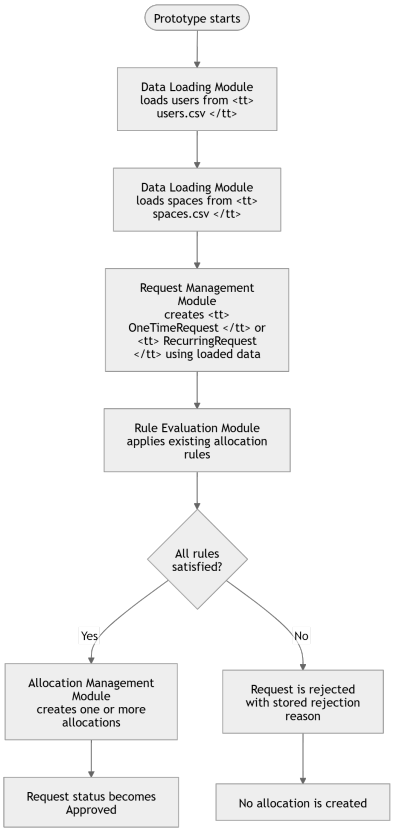

Figure 2: Updated Workflow Diagram

8. Prototype Implementation

To validate the design of this iteration, the C++ prototype was extended by adding a simple `DataLoader` component that reads users and spaces from external CSV files. The prototype now loads initial data from file before creating requests and testing the allocation logic.

The allocation service continues to support both one-time and recurring requests. All previously implemented rules remain unchanged. The difference in this iteration is that the user and space objects used in the requests are no longer hardcoded in `main`, but are instead loaded from external data files.

If the files are successfully loaded, the prototype continues normal allocation processing. If loading fails, the prototype reports an initialization error.

9. Summary

This eleventh implementation iteration improves the prototype by introducing external CSV-based initial data loading while preserving the existing request types, space types, and allocation logic. The system now separates sample data from the main program logic,

making the prototype easier to maintain and extend.

This iteration demonstrates controlled iterative development by improving the realism of the prototype without adding major algorithmic complexity. Future iterations can continue this approach by loading additional data such as existing allocations or requests from external files.